



Práctica

Presentación

Por fin llegamos al final del curso. Esta práctica servirá para sintetizar todos los conocimientos del curso y ampliarlos con el diseño de circuitos más complejos. En esta práctica se os pedirá el diseño de un circuito a partir de un grafo de estados y el diseño de una máquina de estados en concreto. Por la primera parte, tendréis que aplicar los conocimientos que habéis adquirido en este curso, como por ejemplo los mapas de Karnaugh o los sistemas de representación. Por lo tanto, se recomienda que repaséis los anteriores módulos de los materiales.

Competencias

- Conocer la organización general de un computador como circuito digital y Conocer la arquitectura de Von Neumann.

Objetivos

- Conocer varios modelos de las máquinas de estados y de las arquitecturas de controlador con camino de datos.
- Haber adquirido una experiencia básica en la elección del modelo de máquina de estados más adecuado para la resolución de un problema concreto.
- Ser capaz de diseñar circuitos secuenciales a partir de grafos de transiciones de estados.

Recursos

Los recursos que se recomienda utilizar para esta PEC son los siguientes:

Básicos: El módulo 5 de los materiales.

Complementarios: VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- **Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.**
- **La valoración está indicada en cada uno de los subapartados**



Uso de herramientas de IA

En esta actividad no está permitido el uso de herramientas de inteligencia artificial. En el plan docente y en el [web sobre integridad académica y plagio](#) de la UOC encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.

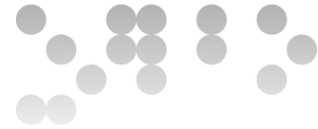
Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado debéis dirigiros al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF utilizando una de las plantillas entregadas conjuntamente con este enunciado.
- Se debe entregar a través de la aplicación de **Entrega de la Actividad** correspondiente del apartado **Contenidos** de vuestra aula.
- La fecha límite de entrega es el **18 de Diciembre** (a las 24 horas).

PRIMERA PARTE [35 %]

Los faros marítimos realizan una secuencia de encendidos y apagones de la luz de forma que permiten ser identificados a partir del ciclo de esta secuencia. Así, además de servir para indicar la posición de la costa, también se pueden usar para identificar de qué faro se trata y, por lo tanto, indirectamente aportar información de la posición marítima del barco. En la tabla siguiente se puede ver una tabla de faros de las Islas Baleares con sus códigos de identificación y las descripciones de las secuencias.

Posición	Nombre del FARO	Código identidad (hex)	Descripción
0h	Cabo de Formentor	32900E0296	Grupo de 4 relámpagos de 0,5 segundos cada uno que se repiten cada 20 segundos
1 h	Punta Avanzada	32950E0298	Grupo de dos apagones de 0,5 segundos cada uno que se repiten cada 8 segundos
2 h	Cabo del Pinar	33090E0303	Grupo de 3 relámpagos de 0,5 segundos cada uno que se repiten cada 9 segundos
3h	Isla Aucanada	33100E0304	1 relámpago de 0,5 segundos repetido cada 5 segundos
4 h	Capdepera	33350E0308	Dos grupos de relámpagos de 0,5 segundos, agrupados en grupos de 2 y 3 relámpagos con 2 segundos de oscuridad entre cada grupo. Se repiten cada 20 segundos
5h	Botafoc	32050E0264	1 apagón de duración 0,5 segundos repetido cada 7 segundos
6h.	Punta de Sa Farola	36470E0344	Dos grupos de apagones de 0,5 segundos, agrupadas en 2 y 3 apagones con 2 segundos de luz entre cada grupo. Se repiten cada 20 segundos



Se pide que diseñéis una máquina de estados algorítmica que implemente la identificación del faro. Para esto, debéis de tener presente la comunicación entre los diferentes módulos disponibles que podéis observar en la Fig.1:

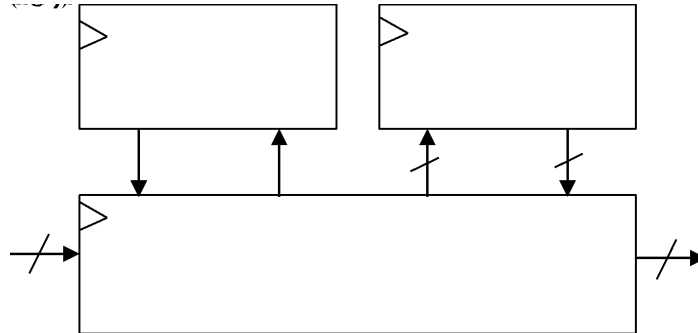


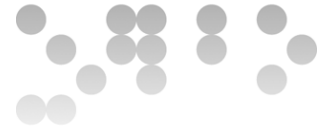
Fig. 1, Comunicación entre los módulos de búsqueda de identificación del faro.

El funcionamiento del controlador será el siguiente:

- Recibe una entrada D de 4 bits que contiene el valor de cada dígito del código de identificación del faro de forma secuencial empezando por el dígito más significativo;
- Genera una salida F de 3 bits cuando el código de entrada es el de un identificador de faro conocido indicando la posición del faro dentro de la tabla, de lo contrario F toma el valor 7h.

Para realizar esta tarea:

- El módulo de control CM controla el inicio y la finalización de la identificación del faro. Por lo tanto, nuestro módulo de identificación LHF empezará la búsqueda en el ciclo que la señal de inicio (*start*) se active, es decir, cuando $s = 1$.
- Mientras realizamos la búsqueda, la señal de indicación de ocupado r (de *ready*) se deberá poner a 0 y volverla a activar al finalizar la búsqueda.
- En cada ciclo posterior, el módulo LHF irá recibiendo por la entrada D un dígito hexadecimal del identificador que se tiene que buscar. El orden de entrada de los dígitos es desde el dígito más significativo hasta el menos significativo. Por simplicidad del sistema, se supone que no hay ninguna pérdida de información y que se reciben los 10 dígitos de forma correcta y secuencialmente.
- La ROM adicional almacena los identificadores de faro codificados por la posición indicada en la tabla anterior (notad que el hecho de almacenar la información por posición reduce significativamente la dimensión de la ROM). Para interactuar con ella, utilizaremos la señal RL para indicar la dirección que queremos leer y recibiremos en el ciclo siguiente el identificador de faro asociado a la dirección en la señal $CODE$.
- Finalmente, el módulo LHF generará una salida F de 3 bits:
 - Si el código es correcto la salida F tomará por valor la posición del faro correcto.
 - De lo contrario, si el código es incorrecto la salida F tomará por valor 7h.



El funcionamiento que se tiene que implementar con el diagrama de estados de la ASM se divide en dos grandes grupos de tareas. La primera parte se tiene que encargar de ir recibiendo los valores de la variable *D*. La segunda parte tiene que buscar secuencialmente en todas las posiciones de la ROM si está almacenado el valor que se acaba de recibir.

Inicialmente, es necesario esperar que la variable *s*, se *active*, y hasta que esta no sea *s=1*, permanecer el estado WAIT. En este estado, inicializamos variables como la variable para el contador de caracteres *D* recibidos (*NUM_DIG*) a valor 0 y la variable *F*. El resto de las inicializaciones se hacen en el estado INIT. Se inicializa la variable que se utilizará para ir iterando sobre las direcciones de la ROM (*DIR*) a la primera posición 0, la variable que almacena todos los valores *D* recibidos (*COD_LH*) con el primer valor de *D* recibo, y el valor de la variable que indica que estamos actualmente haciendo una búsqueda (*r*) con el valor 0 indicando que la búsqueda está activa.

Hasta que no se hayan recibido los 10 caracteres de (*D*) tenemos que iterar en el estado CONCATENATE_CODE. En este estado se concatenan los valores anteriores de *D*, que se han ido almacenando en *COD_LH*, con el último valor que se ha recibido en *D*. Esto se hace con desplazamientos a la izquierda de cuatro bits (multiplicando el valor por 16) y sumándole el valor de *D*. También se actualiza el contador de caracteres recibidos (*NUM_DIG*).

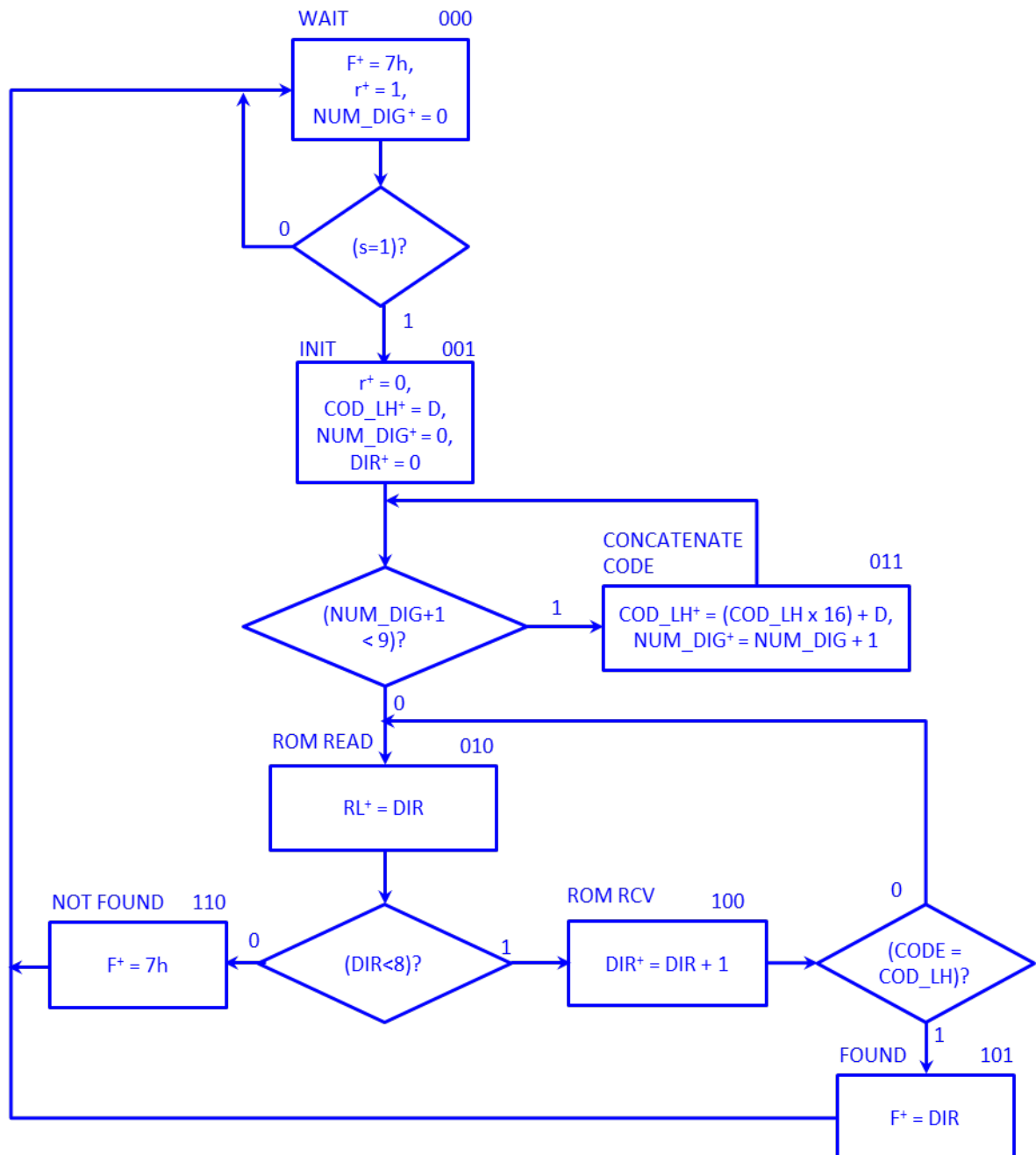
Cuanto *NUM_DIG* llega al valor 10, ya se han recibido todos los dígitos, y la primera fase de recibir el código del faro se ha completado. Ahora se tiene que buscar el valor dentro de la ROM.

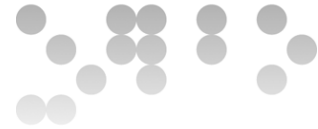
En el estado ROM_READ, indicamos a la ROM la posición que se tiene que consultar. Después de un ciclo, ROM_RCV, que aprovechamos para incrementar la dirección de memoria de la ROM, podemos consultar el contenido de la dirección consultada accediendo a *CODE*. Si el contenido de la ROM es igual al código del faro (*CODE=COD_LH*) lo hemos encontrado, y la búsqueda tiene que pararse y se tiene que devolver la posición de memoria donde está el código en la variable *F*. Esta variable se actualiza en el estado FOUND.

Si el contador de la dirección de memoria de la ROM llega a 8 (*DIR<8* es FALSE), significará que ya hemos consultado todas las posiciones de la ROM y que no hemos encontrado el código del faro almacenado. Entonces, en el estado NOT_FOUND, devolvemos *F* con valor 7h.

Podríamos indicar que se ha acabado la búsqueda en FOUND y NOT_FOUND cambiando el valor de *r=1*, o bien hacerlo en el estado de espera WAIT hasta que llegue otra búsqueda.

La figura siguiente muestra la ASM correspondiente a este comportamiento. Notad que puede haber soluciones alternativas dependiendo de ciertas suposiciones, como por ejemplo, tiempos de espera entre la activación de la señal *s* y la recepción del primer valor de *D*, o bien, tiempo de espera entre la recepción de la última señal de *D* y la búsqueda en la ROM.





SEGUNDA PARTE [30 %]

Cada faro tiene una secuencia lumínica única. En este caso, mediante el identificador anterior se puede acceder a una ROM 8 x 12 donde se puede obtener esta secuencia. Para generar estas secuencias de forma lumínica se usa una EFSM que a partir de una secuencia indica mediante unas señales I y s_i a un módulo externo generador de secuencias lumínicas lo que debe hacer. También se utiliza un temporizador externo para contar segundos de espera entre repeticiones (controlado por la señal T para indicar los segundos de espera y la señal s_T para activar el temporizador). Las secuencias lumínicas se codifican de la siguiente forma:

Agrupaciones (A)	# Encendidos o # Apagones primer grupo (G1)	# Encendidos o # Apagones segundo grupo (G2)	# Segundos entre repeticiones (R)
2 bits	2 bits	3 bits	5 bits

A partir de las secuencias lumínicas de la lista de faros se determinan los posibles valores posibles de las agrupaciones:

Agrupaciones	Significado.
00	Secuencia sin agrupaciones
01	Secuencia con dos agrupaciones luminosas
10	Secuencia con dos agrupaciones de apagones
11	Combinación no permitida

Por ejemplo, la codificación de los siguientes faros sería:

Cabo del Pinar	Grupo de 3 relámpagos de 0,5 segundos cada uno que se repiten cada 9 segundos	00 11 000 01001
Isla Aucanada	Grupo de 1 relámpago de 0,5 segundos repetido cada 5 segundos	00 01 000 00101
Punta de Sa Farola	Dos grupos de apagones de 0,5 segundos, agrupados en 2 y 3 apagones con 2 segundos de luz entre cada grupo. Se repiten cada 20 segundos	10 10 011 10100

Por lo tanto, la EFSM recibe la codificación de las secuencias mediante 4 señales diferentes: A , para el tipo de agrupaciones, $G1$, para el número de encendidos o apagones del primer grupo, $G2$ para el número de encendidos o apagones del segundo grupo y R para el número de repeticiones. A partir de esta información, genera de forma ininterrumpida las señales I y s_i .

Esta sería el EFSM de la máquina que recibe la posición del faro identificado y genera la secuencia lumínica (Los arcos no etiquetados indican transiciones incondicionales, que se efectúan siempre).

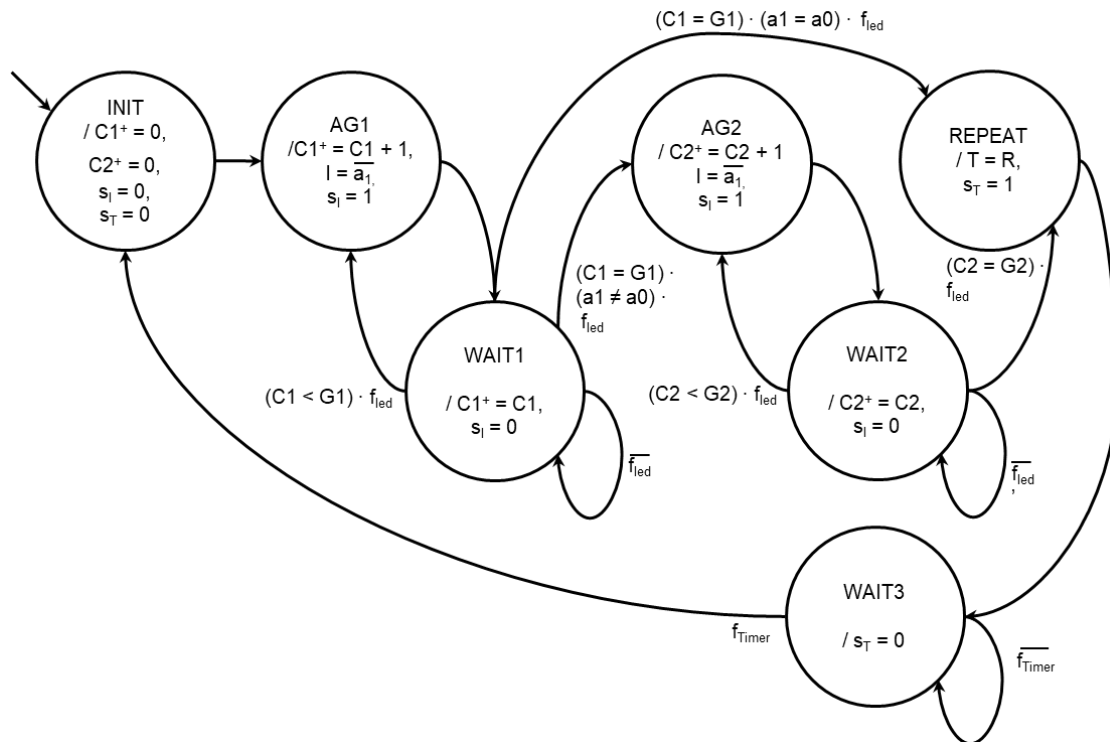
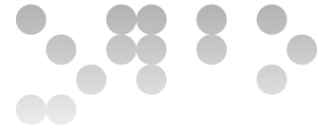
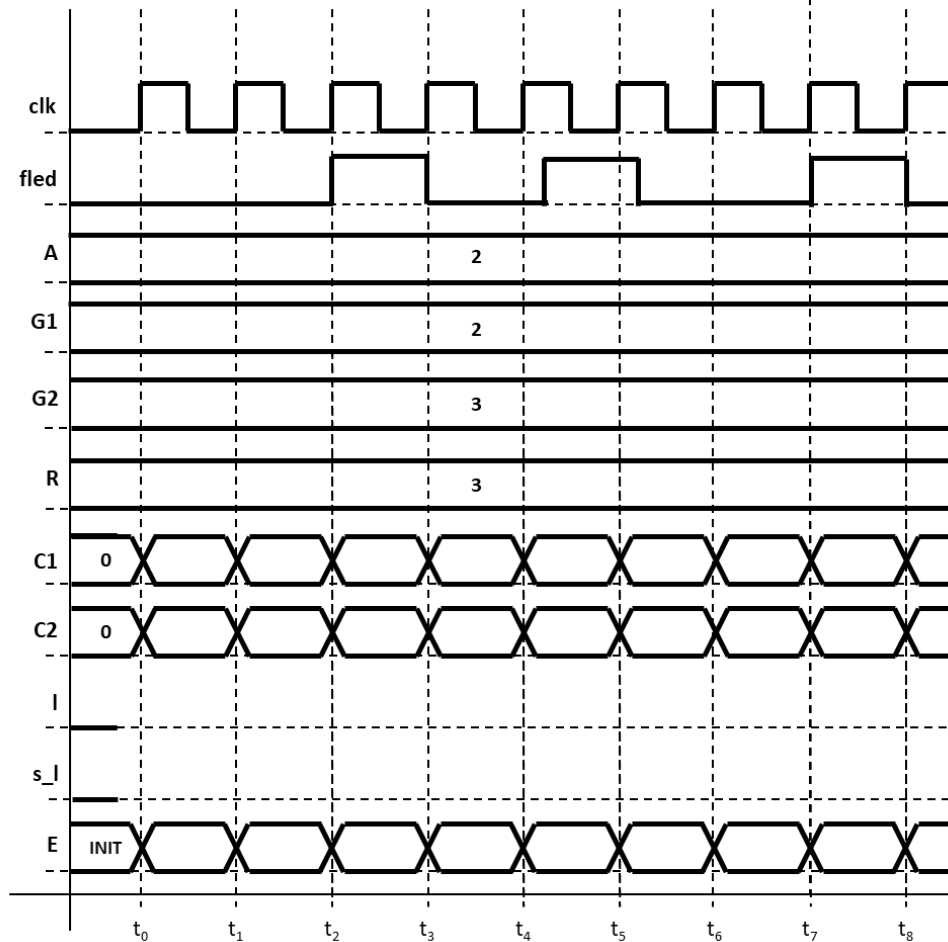


Fig. 2. EFSM del módulo de generación de secuencias lumínicas.

Se pide que:

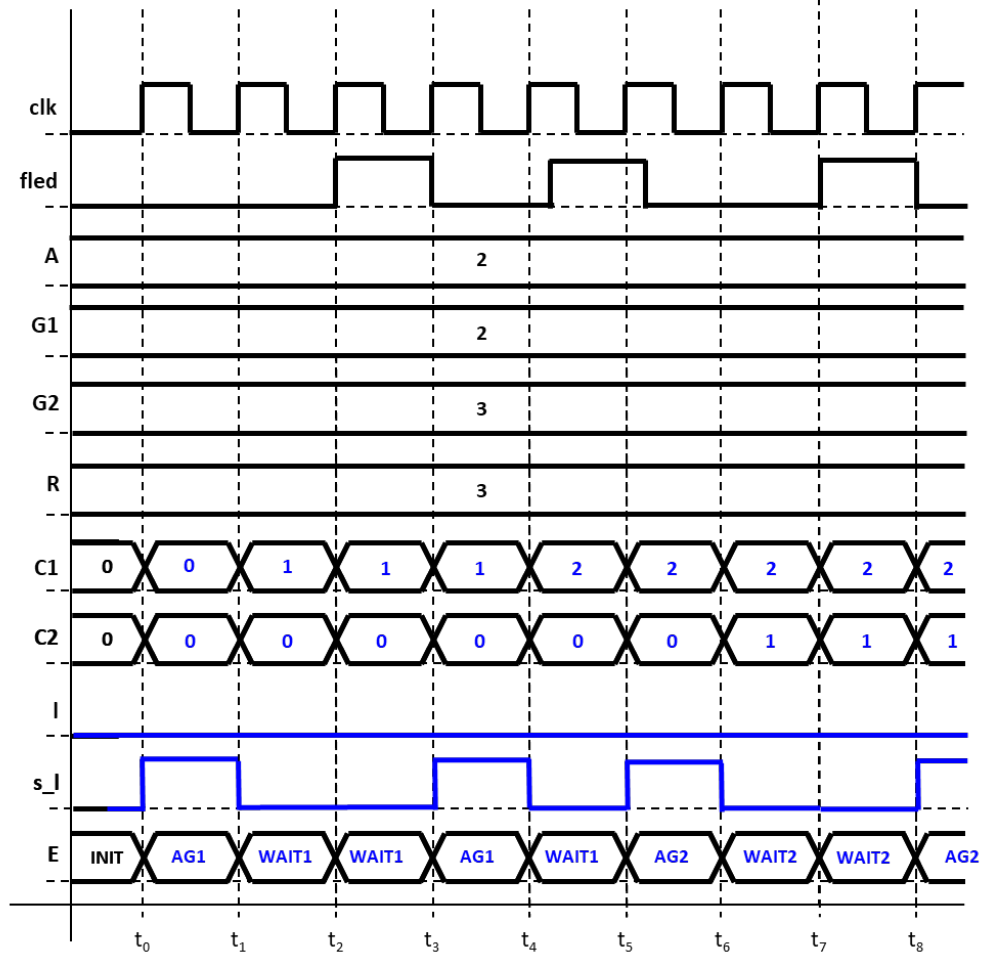
a) [15 %] A partir del grafo del EFSM de la Fig.2, completad el cronograma siguiente:

Nota: Tenéis el ejercicio disponible a VerilChart, donde el estado es E y los valores de las señales numéricas se representan en hexadecimal.



Haciendo el seguimiento del grafo, a partir de las condiciones iniciales que nos indica el cronograma, podremos saber cómo evolucionan el estado, las variables s_i y I , y las señales $C1$ y $C2$. En cada ciclo de reloj se calcula, según la situación que determina el estado, el siguiente valor que toman las variables $C1$ y $C2$. Recordad que estas variables, se actualizarán al final del ciclo, en el flanco ascendente, con los valores de las variables justo antes del flanco. Por otro lado, los valores de las señales I y s_i dependen directamente del estado en que se encuentra la máquina, y sus valores se determinan en el mismo ciclo teniendo en cuenta el estado actual.

Con estas indicaciones, y siguiendo el diagrama de estados, el cronograma queda de la manera siguiente:



- b) [15 %] Obtened las tablas de transiciones y de salidas de lo EFSM de la Fig.2, así como las codificaciones binarias de estados y salidas correspondientes.

En el grafo están las señales de entrada f_{led} y f_{Timer} , que condicionan las transiciones. También están las variables $C1$ y $C2$, que determinarán las transiciones en función de sus valores. En dicho grafo, las condiciones de los valores de $C1$ y $C2$ son $(C_x < G_x)$ y $(C_x = G_x)$, dónde x puede tomar el valor 1 o 2. Estas dos condiciones se pueden expresar usando la de igualdad, porque las transiciones solo están condicionadas en que el valor de C_x sea menor a G_x , o en que el valor de C_x sea igual a G_x . Por eso, consideraremos únicamente las variables $(C1=G1)$ y $(C2=G2)$ con valores posibles 0 (*false*) o 1 (*true*).



Estado actual	Entrada	Estado siguiente
INIT	x	AG1
AG1	x	WAIT1
WAIT1	$(f_{led})'$	WAIT1
WAIT1	$(C1=G1)' \cdot (f_{led})$	AG1
WAIT1	$(C1=G1) \cdot (a_1=a_0)' \cdot (f_{led})$	AG2
WAIT1	$(C1=G1) \cdot (a_1=a_0) \cdot (f_{led})$	REPEAT
AG2	x	WAIT2
WAIT2	$(f_{led})'$	WAIT2
WAIT2	$(C2=G2)' \cdot (f_{led})$	AG2
WAIT2	$(C2=G2) \cdot (f_{led})$	REPEAT
REPEAT	x	WAIT3
WAIT3	$(f_{timer})'$	WAIT3
WAIT3	(f_{timer})	INIT

Estado	Salidas
INIT	$C1^+=0, C2^+=0, s_i=0, s_T=0$
AG1	$C1^+=C1+1, l=a_1', s_i=1$
WAIT1	$C1^+=C1, s_i=0$
AG2	$C2^+=C2+1, l=a_1', s_i=1$
WAIT2	$C2^+=C2, s_i=0$
REPEAT	$T=R, s_T=1$
WAIT3	$s_T=0$

Para la codificación binaria, hay que asignar en cada estado un código binario individual. En este caso, se sigue el criterio más habitual, que es el de la numeración binaria, desde el 0 para el estado inicial hasta el número de estados que haya menos 1: 000 para INIT, 001 para AG1, 010 para WAIT1, 011 para AG2, 100 para WAIT2, 101 para REPEAT, y 110 para WAIT3.

Para elegir el valor siguiente que tenemos que asignar a la variable *C1* necesitamos un selector, que denominaremos *s_C1*. En la tabla siguiente tenemos los valores entre los que se elige y una posible codificación binaria de cada uno de ellos:

Operación	Selector <i>s_C1</i>	Efecto sobre la variable
$C1^+=0$	00	Inicializarla a 0
$C1^+=C1+1$	01	Sumarle 1
$C1^+=C1$	10	Mantener el valor

De forma similar, usaremos un selector, *s_C2*, para seleccionar el valor siguiente de la variable *C2*:

Operación	Selector <i>s_C2</i>	Efecto sobre la variable
$C2^+=0$	00	Inicializarla a 0
$C2^+=C2+1$	01	Sumarle 1



Operación	Selector s_C2	Efecto sobre la variable
C2+ = C2	10	Mantener el valor

Finalmente, hace falta implementar el comportamiento de las variables I y T . En lugar de utilizar selectores con señales de salida adicionales, podemos utilizar unos MUXs cuyas señales de selección sean las variables s_T y s_I . Observad que $T=R$ siempre que $s_T=1$, y $I=a_1'$ siempre que $s_I=1$. Entonces,

Operación	s_I	Efecto sobre la variable
$I = a_1'$	1	La salida I coge el valor inverso a a_1
$I=0$	0	La salida I coge el valor 0

y

Operación	s_T	Efecto sobre la variable
$T = R$	1	La salida T coge el valor de R
$T=0$	0	T coge valor 0.

Los valores de las salidas s_I y s_T se determinan directamente a partir del estado.

Finalmente, las tablas de transiciones y de salidas con la codificación binaria correspondiente son:

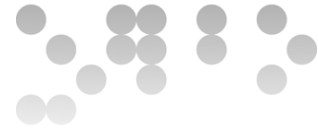
Estado actual		Entrada					Estado ⁺
Símbolo	$q_2q_1q_0$	(C1=G1)	($a_1=a_0$)	(C2=G2)	f_{led}	f_{timer}	$q_2^+q_1^+q_0^+$
INIT	000	x	x	x	x	x	001
AG1	001	x	x	x	x	x	010
WAIT1	010	x	x	x	0	x	010
WAIT1	010	0	x	x	1	x	001
WAIT1	010	1	0	x	1	x	011
WAIT1	010	1	1	x	1	x	101
AG2	011	x	x	x	x	x	100
WAIT2	100	x	x	x	0	x	100



Estado actual		Entrada					Estado*
Símbolo	$q_2q_1q_0$	(C1=G1)	(a1=a0)	(C2=G2)	f_{led}	f_{timer}	$q_2^+q_1^+q_0^+$
WAIT2	100	x	x	0	1	x	011
WAIT2	100	x	x	1	1	x	101
REPEAT	101	x	x	x	x	x	110
WAIT3	110	x	x	x	x	0	110
WAIT3	110	x	x	x	x	1	000

Estado		Salidas			
Símbolo	$q_2q_1q_0$	s_C1	s_C2	s_l	s_T
INIT	000	00	00	0	0
AG1	001	01	x	1	x
WAIT1	010	10	x	0	x
AG2	011	x	01	1	x
WAIT2	100	x	10	0	x
REPEAT	101	x	x	x	1
WAIT3	110	x	x	x	0

Notad que según la codificación elegida por los estados y por los selectores, el resultado de la tabla anterior puede variar.



TERCERA PARTE [35 %]

El EFSM siguiente se corresponde con el funcionamiento de una máquina de café sencilla. Esta máquina de café tiene la siguiente señal de entrada:

- *N/D (NORMAL/DOUBLE)*: señal que proviene de un pulsador externo para indicar que el café se tiene que servir normal (valor 0) o doble (valor 1). Este pulsador realmente debería tener tres posiciones, para esperar la selección del tipo de café. Pero para simplificar el ejercicio, supondremos que la máquina de café no tiene un estado de espera activa, es decir, una vez que el agua esté caliente, se servirá el café según si se quiere normal o doble.

Y las siguientes señales de salida:

- *heat*: para controlar el calefactor de agua ('1' enciende el calefactor, '0' lo apaga),
- *water*: para controlar la válvula de salida del café ('1' deja salir el agua, '0' detiene la salida de agua),
- *light*: para controlar el piloto que indica si la temperatura del agua es la adecuada ('1' enciende el piloto, '0' lo apaga)

La máquina tiene dos funciones principales que consisten en mantener el agua caliente y servir café en función del pulsador externo. La temperatura del agua se controla mediante la variable temperatura T , mientras que servir el café se realiza mediante un contador interno que según la selección tarda más o menos tiempo en servirlo.

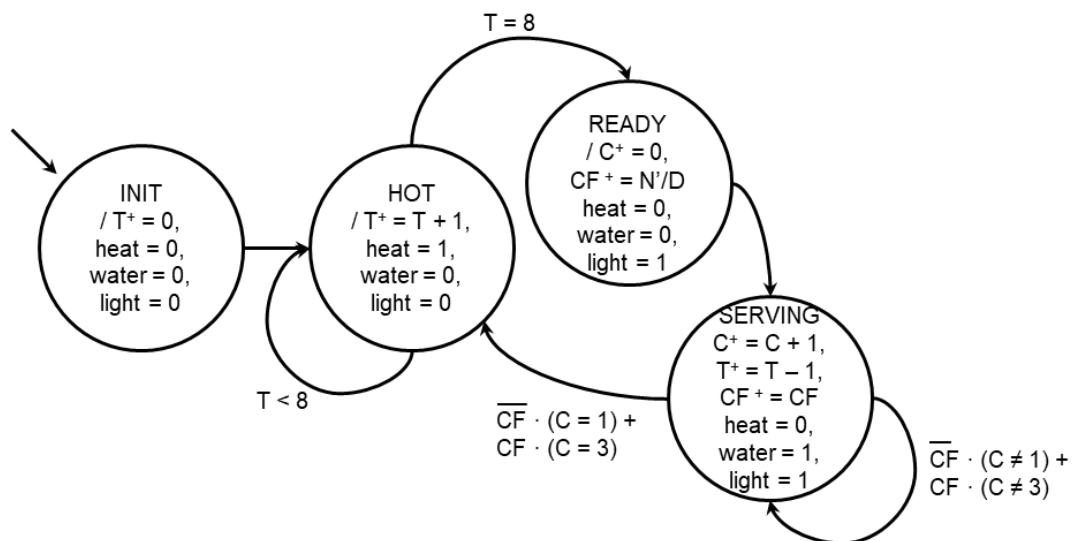
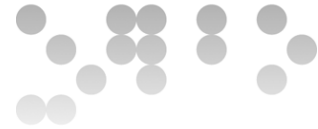


Fig. 3. EFSM de control de la máquina cafetera.

Se pide que:

- [15 %] A partir de las tablas de transiciones y de salidas del grafo de la Fig. 3 que se dan a continuación, implementéis la unidad de control usando ROM y los registros, los bloques combinacionales y las puertas lógicas que creáis convenientes. Especificad y justificad las dimensiones de la/las ROM que usáis e indicad el contenido en binario, indicando a que corresponde cada bit, y en hexadecimal.



Estado actual	Entradas				Estado ⁺
	T=8	C=1	C=3	CF	
INIT	x	x	x	x	HOT
HOT	0	x	x	x	HOT
HOT	1	x	x	x	READY
READY	x	x	x	x	SERV
SERV	x	0	x	0	SERV
SERV	x	x	0	1	SERV
SERV	x	1	x	0	HOT
SERV	x	x	1	1	HOT

Estado		Salidas					
Símbolo	q ₁ q ₀	heat	water	light	s_C	s_T	s_CF
INIT	00	0	0	0	0	00	0
HOT	01	1	0	0	0	01	0
READY	10	0	0	1	0	11	1
SERV	11	0	1	1	1	10	0

Nota: Tenéis el ejercicio disponible a VerilCIRC y, si queréis comprobar previamente la corrección de la ROM válida con palabras de 9 bits, la tenéis disponible en VerilChart. Tened en cuenta que hay otras opciones igualmente válidas con ROM de diferentes dimensiones. En este entorno, el nombre de la señal de estado es E , y el de la entrada de control ($T=8$) es $T8$, ($C=1$) es $C1$, ($C=3$) es $C3$ y N/D es ND .

Para implementar la unidad de control con una ROM hace falta, primero, determinar el tamaño y, después, calcular el contenido a partir de la tabla de transiciones correspondiente.

El bus de direcciones de la ROM tiene que ser de 6 bits, puesto que la dirección se forma con los bits que codifican el estado (son 2, puesto que hay cuatro estados) y los de las señales de entrada (4 más por las señales $T=8$, $C=1$, $C=3$, CF). Por lo tanto, la ROM tiene que disponer de un total de $2^6 = 64$ posiciones de memoria.

En cada posición se almacena el código del estado siguiente y el valor, en el estado actual, de cada uno de las señales de salida. Por un lado, hay tres salidas directas (*heat*, *water*, *light*). Además, se tienen que definir las salidas que controlarán las operaciones a realizar con las variables C , T y CF .

Para elegir el valor siguiente que tenemos que asignar a la variable C necesitamos un selector, que denominaremos s_C . En la tabla siguiente tenemos los valores entre los que se elige y una posible codificación binaria de cada uno de ellos:

Operación	Selector s_C	Efecto sobre la variable
$C^+ = 0$	0	Inicializarla a 0
$C^+ = C+1$	1	Sumarle 1



De forma similar, usaremos un selector, s_T , para seleccionar el valor siguiente de la variable T :

Operación	Selector s_T	Efecto sobre la variable
$T^+ = 0$	00	Posarla a 0
$T^+ = T+1$	01	Sumarle 1
$T^+ = T-1$	10	Restarle 1
$T^+ = T$	11	Mantener el valor actual de T

Finalmente, usaremos un selector s_CF para guardar el tipo de café que se debe realizar, de forma que aunque se pulse el pulsador durante la realización del café no afecte al funcionamiento de la máquina.

Operación	Selector s_CF	Efecto sobre la variable
$CF^+ = CF$	0	Mantener el valor actual de CF
$CF^+ = N/D$	1	Guardar el valor de la entrada N/D

Por lo tanto, hacen falta 2 bits para el estado y harían falta 7 bits para las salidas (*heat*, *water*, *light*, s_C , s_T , s_CF), lo que suma un total de 9 bits.

Así pues, si se usa una única ROM, tendrá que ser de 64x9 bits. El contenido de la ROM se determina a partir de la tabla de transiciones y la de salidas (los casos con *don't cares* los hemos considerado como valores 0):

	Posición de memoria				Contenido							Codificación hexadecimal
q_1q_0	T=8	C=1	C=3	CF	$q_1^+q_0^+$	heat	water	light	s_C	s_T	s_CF	
00	0	0	0	0	01	0	0	0	0	00	0	080h
00	0	0	0	1	01	0	0	0	0	00	0	080h
00	0	0	1	0	01	0	0	0	0	00	0	080h
00	0	0	1	1	01	0	0	0	0	00	0	080h
00	0	1	0	0	01	0	0	0	0	00	0	080h
00	0	1	0	1	01	0	0	0	0	00	0	080h
00	0	1	1	0	01	0	0	0	0	00	0	080h
00	0	1	1	1	01	0	0	0	0	00	0	080h



q ₁ q ₀	Posición de memoria				Contenido							Codificación hexadecimal
	T=8	C=1	C=3	CF	q ₁ *q ₀ *	heat	water	light	s_C	s_T	s_CF	
00	1	0	0	0	01	0	0	0	0	00	0	080h
00	1	0	0	1	01	0	0	0	0	00	0	080h
00	1	0	1	0	01	0	0	0	0	00	0	080h
00	1	0	1	1	01	0	0	0	0	00	0	080h
00	1	1	0	0	01	0	0	0	0	00	0	080h
00	1	1	0	1	01	0	0	0	0	00	0	080h
00	1	1	1	0	01	0	0	0	0	00	0	080h
00	1	1	1	1	01	0	0	0	0	00	0	080h
01	0	0	0	0	01	1	0	0	0	01	0	0C2h
01	0	0	0	1	01	1	0	0	0	01	0	0C2h
01	0	0	1	0	01	1	0	0	0	01	0	0C2h
01	0	0	1	1	01	1	0	0	0	01	0	0C2h
01	0	1	0	0	01	1	0	0	0	01	0	0C2h
01	0	1	0	1	01	1	0	0	0	01	0	0C2h
01	0	1	1	0	01	1	0	0	0	01	0	0C2h
01	0	1	1	1	01	1	0	0	0	01	0	0C2h
01	1	0	0	0	10	1	0	0	0	01	0	142h
01	1	0	0	1	10	1	0	0	0	01	0	142h
01	1	0	1	0	10	1	0	0	0	01	0	142h
01	1	0	1	1	10	1	0	0	0	01	0	142h
01	1	1	0	0	10	1	0	0	0	01	0	142h
01	1	1	0	1	10	1	0	0	0	01	0	142h

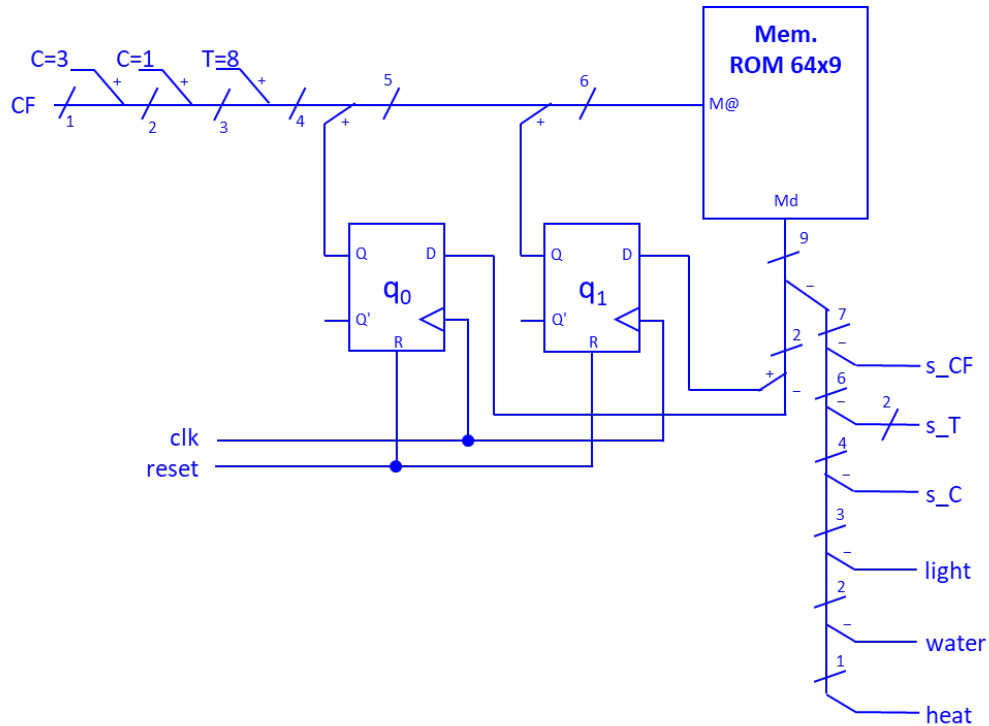


q ₁ q ₀	Posición de memoria				Contenido							Codificación hexadecimal
	T=8	C=1	C=3	CF	q ₁ *q ₀ *	heat	water	light	s_C	s_T	s_CF	
01	1	1	1	0	10	1	0	0	0	01	0	142h
01	1	1	1	1	10	1	0	0	0	01	0	142h
10	0	0	0	0	11	0	0	1	0	11	1	197h
10	0	0	0	1	11	0	0	1	0	11	1	197h
10	0	0	1	0	11	0	0	1	0	11	1	197h
10	0	0	1	1	11	0	0	1	0	11	1	197h
10	0	1	0	0	11	0	0	1	0	11	1	197h
10	0	1	0	1	11	0	0	1	0	11	1	197h
10	0	1	1	0	11	0	0	1	0	11	1	197h
10	0	1	1	1	11	0	0	1	0	11	1	197h
10	1	0	0	0	11	0	0	1	0	11	1	197h
10	1	0	0	1	11	0	0	1	0	11	1	197h
10	1	0	1	0	11	0	0	1	0	11	1	197h
10	1	0	1	1	11	0	0	1	0	11	1	197h
10	1	1	0	0	11	0	0	1	0	11	1	197h
10	1	1	0	1	11	0	0	1	0	11	1	197h
10	1	1	1	0	11	0	0	1	0	11	1	197h
10	1	1	1	1	11	0	0	1	0	11	1	197h
11	0	0	0	0	11	0	1	1	1	10	0	1BCh
11	0	0	0	1	11	0	1	1	1	10	0	1BCh
11	0	0	1	0	11	0	1	1	1	10	0	1BCh
11	0	0	1	1	01	0	1	1	1	10	0	0BCh
11	0	1	0	0	01	0	1	1	1	10	0	0BCh
11	0	1	0	1	11	0	1	1	1	10	0	1BCh
11	0	1	1	0	01	0	1	1	1	10	0	0BCh
11	0	1	1	1	01	0	1	1	1	10	0	0BCh
11	1	0	0	0	11	0	1	1	1	10	0	1BCh
11	1	0	0	1	11	0	1	1	1	10	0	1BCh
11	1	0	1	0	11	0	1	1	1	10	0	1BCh
11	1	0	1	1	01	0	1	1	1	10	0	0BCh
11	1	1	0	0	01	0	1	1	1	10	0	0BCh
11	1	1	0	1	11	0	1	1	1	10	0	1BCh
11	1	1	1	0	01	0	1	1	1	10	0	0BCh
11	1	1	1	1	01	0	1	1	1	10	0	0BCh

La figura siguiente muestra la implementación con la ROM correspondiente. La entrada M@ de la ROM se configura con el valor del estado actual (los 2 bits de más peso) y



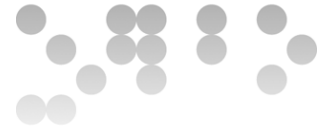
los valores de las señales de entrada (los bits de menos peso). Para guardar el estado actual se usan 2 biestables.



Con dos ROMs, una para la tabla de transiciones y otra para la de salidas, la primera sería de 64x2 (es la misma que el anterior, con las palabras de memoria que solo contienen la codificación del estado siguiente) y la segunda, de 4x7, tal como se muestra a continuación.

Posición de memoria	Contenido						Codificación hexadecimal
q ₁ q ₀	heat	water	light	s_C	s_T	s_CF	
00	0	0	0	0	00	0	00h
01	1	0	0	0	01	0	42h
10	0	0	1	0	11	1	17h
11	0	1	1	1	10	0	3Ch

El circuito correspondiente queda como se muestra a continuación.



Se ha decidido utilizar 3 bits para implementar la variable C porque, una vez que se produce la condición $C=3$, se realiza la transición al estado HOT y se actualiza el valor de C a 4, ya que las variables se actualizan en el camino de datos en el ciclo siguiente. Se podría haber decidido utilizar solo 2 bits para esta variable C y haber permitido el desbordamiento al valor 0 de dicha variable una vez que se incrementa el valor 3 al volver al estado HOT, ya que, en este estado, el valor de la variable C no tiene ninguna relevancia.

De manera similar, la variable T debe tener una medida de 4 bits, puesto que es un contador de valor máximo 8 (1000_2). Las operaciones sobre esta variable son inicializarla a 0, mantener su valor, incrementarlo en 1 unidad, o decrementarlo en una unidad. Por eso, y con un MUX de dos entradas de selección controladas por la salida s_T de la unidad de control, y tal como se ha establecido en el apartado anterior, seleccionarán entre las cuatro posibles operaciones. Es decir, el valor 0 decimal de cuatro bits, el valor actual de T almacenado en el registro, o los resultados de sumar el valor 1 al valor actual de T o restarle 1. La resta se ha implementado como una suma en complemento a 2 del valor T con el valor -1. Por eso ha hecho falta incrementar en un bit la precisión de los operandos, y como siempre el resultado de la resta será menor al valor de T inicial, se puede reducir el resultado de esta resta a cuatro bits sin ningún problema.

Finalmente, la variable CF únicamente tiene que guardar el valor de la entrada N'/D. Por esta razón, se necesitará un biestable para guardar este bit. La escritura estará controlada por un MUX y el selector s_CF .

